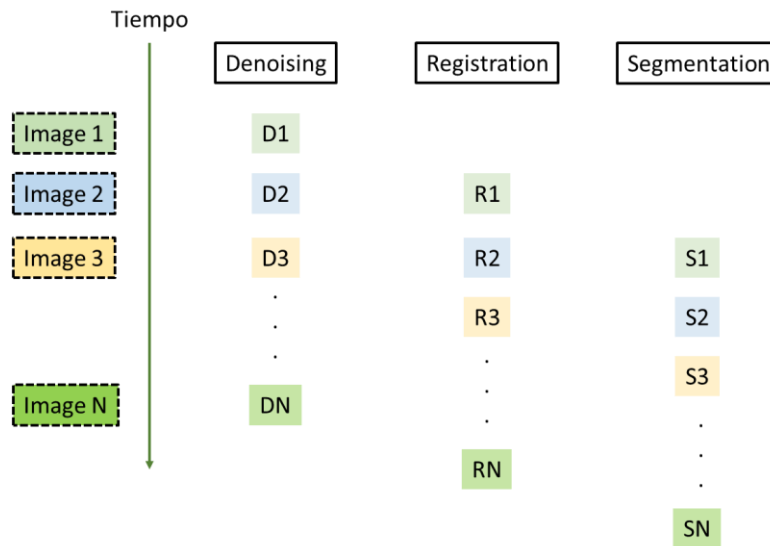
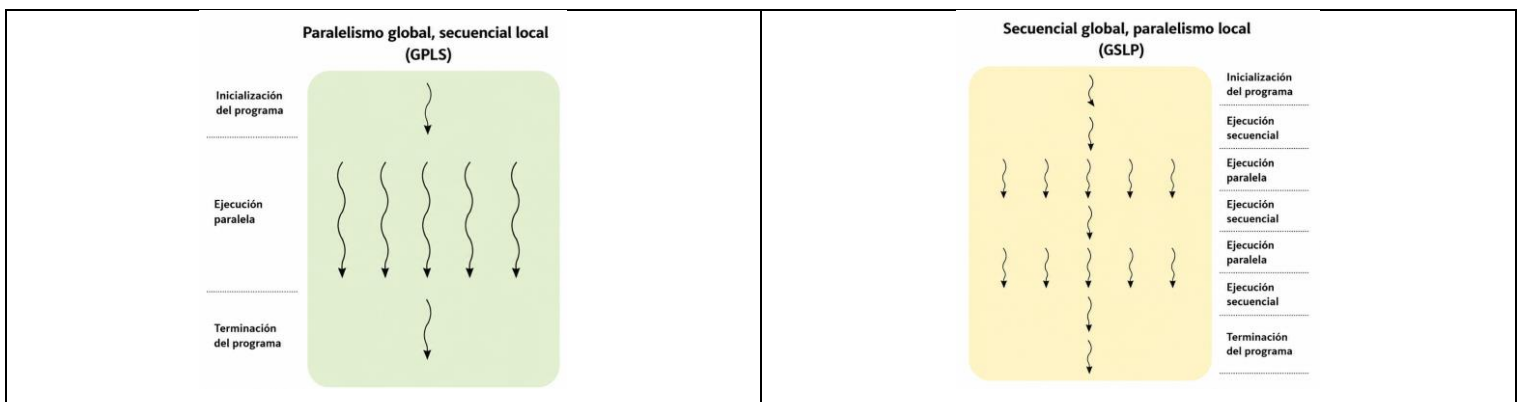


Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Computación Paralela y Distribuida
Parcial No. 2, Parte 2

6. En paralelismo funcional se utiliza programación funcional. Responda las siguientes preguntas en relación con la teoría de categorías.
- ¿Qué es un morfismo?
 - ¿Qué hace un functor a un morfismo?
 - ¿En qué parte del material publicado en el classroom del curso se presentan estos dos conceptos?
7. Con base en el documento de trabajo publicado en <https://www.arcesio.net/divulgacion/Desagregacion.pdf> ¿En qué patrón de paralelismo se basa Apache Spark? Explique su respuesta.
8. Si se tiene una computación donde se hacen tres etapas del procesamiento de imágenes: *Denoising*, *Registration* y *Segmentation*, utilizando un patrón de pipeline
- ¿Cuál es el Work, el SPAN y el paralelismo ideal para procesar N imágenes?
 - Describa cómo se puede utilizar phasers de Java para realizar el procesamiento descrito.



9. Con base en lo visto en clase,
- Describa lo que representan las dos imágenes siguientes.
 - ¿Qué relación tiene el modelo de ejecución de CUDA para GPUs con estas imágenes?



10. En el código mostrado, explique de manera breve cuál es el propósito de las instrucciones de

- a. la línea 3
- b. la línea 12
- c. la línea 22
- d. la línea 23

```
1 #include <iostream>
2 #include <math.h>
3 __global__
4 void add(int n, float *x, float *y)
5 {
6     int index = blockIdx.x * blockDim.x + threadIdx.x;
7     int stride = blockDim.x * gridDim.x;
8     for (int i = index; i < n; i += stride)
9         y[i] = x[i] + y[i];
10 }
11 int main(void) {
12     int N = 1<<20;
13     float *x, *y;
14     int blockSize = 256;
15     int numBlocks = (N + blockSize - 1) / blockSize;
16     cudaMallocManaged(&x, N*sizeof(float));
17     cudaMallocManaged(&y, N*sizeof(float));
18     for (int i = 0; i < N; i++) {
19         x[i] = 1.0f;
20         y[i] = 2.0f;
21     }
22     add<<<numBlocks, blockSize>>>>(N, x, y);
23     cudaDeviceSynchronize();
24     float maxError = 0.0f;
25     for (int i = 0; i < N; i++) {
26         maxError = fmax(maxError, fabs(y[i]-3.0f));
27     }
28     std::cout << "Max error: " << maxError << std::endl;
29     cudaFree(x);
30     cudaFree(y);
31     return 0;
32 }
```